

s4.03 — Uber — matching, live locations, and trip management

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Location ingest and geo index — IC3 baseline](#)
- [Matching, offers, and the trip state machine — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: first-hand report attribution `1p3a x2` is retained in [the bank](#).
 - Two collected reports name this prompt, so it is the most likely business prompt in the loop.
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Uber's implementation or an interviewer's accepted answer.
- OCI framing for a business prompt
 - Control plane: trip and driver state, quotas, pricing configuration, tenant onboarding.
 - Data plane: location ingest, geo index, live-location fan-out.
 - Static stability: an in-progress trip must finish even if the trip database is unavailable.
 - Blast radius: one region or city cell fails alone.
- Why this is hard
 - Naive: a `drivers` table with `lat`, `lon`, a `WHERE distance < r` query, and "assign the first row".
 - First break: hundreds of thousands of location writes per second and a full-table distance scan on every request.
 - Second break: two riders receive the same driver because "nearby and available in the cache" is not an atomic assignment.
 - Third break: reordered or replayed location updates move a driver backward, and a driver that lost connectivity still looks available.
 - Minimum primitives
 - A cell-indexed in-memory latest-location store with freshness expiry.
 - Session and sequence fencing on every location update.
 - An offer with a deadline and a deterministic identity.
 - One conditional transaction that claims the trip and the driver together.
 - A versioned trip state machine with actor-specific transitions.
- 🧠 "The geo index only nominates candidates; the trip transaction is the only thing that assigns a driver."

- Opening script
 - "I will split this into a high-write location data plane and a low-write, transactional trip control plane."
 - "First I confirm the scale of online drivers, the matching policy, and what must keep working when the trip store is down."
 - "Then I show that location ingest is a memory and fan-out problem while matching is a correctness problem at a much lower rate."
 - "The architecture is a fenced location ingest into a cell index, a per-region matcher that issues timed offers, and a trip service whose accept transaction is the single point of assignment."
 - "The deep dives are the geo index with freshness and the offer-accept race across two riders and one driver."
 - "I will close with degradation when the trip store is unavailable, tenant isolation, and where batched matching changes the design."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Location write rate, match rate; API semantics
Architecture	17:00–30:00	Diagram, plane split, one ride-request trace
Deep dive	30:00–45:00	Cell index and fencing; offer-accept transaction; state machine
Operations and tradeoffs	45:00–50:00	Failure table, degraded matching, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Scale?	1M drivers online globally at peak, 100k in the busiest metro, 25M trips/day	Location ingest is the high-write path; matching is two orders of magnitude lower
Location update interval?	Every 4 s while online, 1 s while on a trip	Fixes write rate, freshness window, and fan-out rate
Matching policy?	Nearest-ETA driver, one offer at a time, driver may decline	Sequential offers with a deadline; batched assignment is an IC4 stretch
Who is the tenant?	A city or fleet-operator partition in one deployment	Tenant key on every row; matching never crosses tenants
Consistency of assignment?	Exactly one driver per trip and one active trip per driver	Needs a transactional claim, not a cache flag
What survives a trip-store outage?	In-progress trips continue; new requests fail fast	Device holds trip state; ingest and fan-out do not depend on the trip store
Pricing and payment in scope?	Quote before request; settlement after completion, out of the matching path	Quote has a bounded lifetime; payment is an idempotent downstream operation

- Functional scope
 - Ingest driver locations and keep a fresh nearest-driver index.
 - Quote a ride, accept a ride request, and match it to one available driver.
 - Offer, accept, decline, and time out with bounded retries.
 - Drive the trip through a state machine with actor-specific transitions.

- Show the assigned driver's live location to the rider only.
- Cancel with explicit state-dependent rules and settle after completion.

Requirement	Target	Enforced by
Location write rate	250k/s global, 25k/s per busiest region	In-memory latest-location store sharded by region and cell; no relational write per update
Location freshness	Index entry expires after 30 s without an update	TTL on the latest-location entry; offer re-checks freshness
Time to first offer	p95 < 3 s from request	Cell cover query plus ETA on top 10 candidates only
Time to assignment	p95 < 60 s; NO_DRIVERS after 5 rounds or 90 s	Offer deadline 15 s; bounded rounds per trip
Assignment correctness	0 double-assigned drivers or trips	One conditional transaction plus a partial unique index on active trips per driver
Trip-state availability	99.95% for transitions	Stateless trip API; replicated relational store per region
Live-location latency to rider	p95 < 2 s	Pub/sub keyed by trip; websocket gateway sharded by trip
Static stability	In-progress trips complete during a 30 min trip-store outage	Device-held trip state; completion events queue and replay
Location history retention	30 d, async	Append to a log and object storage, never on the ingest hot path

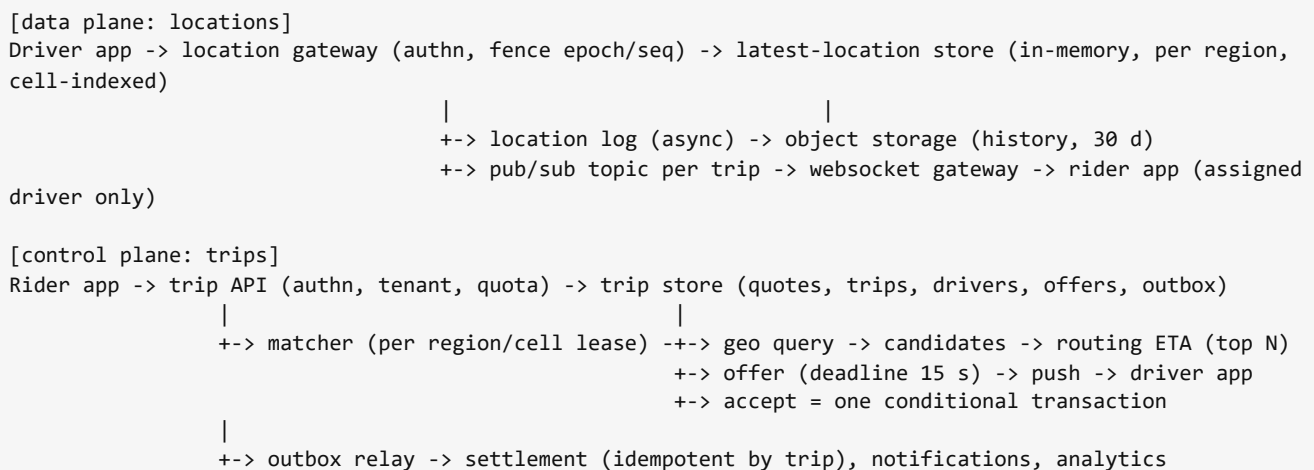
- Invariants
 - A driver holds at most one active trip at any time.
 - A trip has at most one assigned driver, chosen by exactly one committed accept transaction.
 - A location update is applied only if its (session_epoch, seq) is newer than the stored value.
 - A candidate returned by the geo index is never treated as assigned until the trip transaction commits.
 - A trip transition requires the expected version and an actor allowed to perform it.
 - A rider sees only the location of the driver assigned to that rider's active trip.
- Non-goals
 - Route planning and turn-by-turn navigation; the routing service is a dependency with an ETA API.
 - Surge-pricing models; the quote service is an input, not this design.
 - Ride pooling and multi-stop trips in the first release.
 - Fraud detection and driver onboarding.

Capacity and consequence — IC3 baseline

- Location ingest
 - 1M online drivers updating every 4 s is 250k updates/s globally.
 - The busiest metro at 100k drivers is 25k updates/s.
 - Each update is about 120 B, so ingest is roughly 30 MB/s globally, which is trivial network but not a trivial write rate.
 - Decision: latest location lives in an in-memory store sharded by region and cell.
 - A relational row update per location is the wrong tool at this rate.
 - The store is a projection that can be rebuilt from live updates within one freshness window.
- Latest-location state size
 - 1M entries at about 200 B with index membership is about 200 MB.
 - Decision: one region's state fits in memory on a few nodes with a replica.
 - Shard by cell prefix so a hot metro does not sit on one node.
- Location history
 - 250k/s × 120 B ≈ 30 MB/s, about 2.6 TB/day raw, roughly 500 GB/day compressed at an assumed 5×.
 - 30 d retention is on the order of 15 TB.

- Decision: history is an async append to a log and then object storage.
 - It is never on the ingest acknowledgment path.
 - Partition by driver and time bucket for trip replay and disputes.
- Ride requests and matching
 - 25M trips/day $\approx$ 290 requests/s average; a 4 $\times$ peak is about 1.2k/s globally and about 120/s in the busiest metro.
 - One match covers 10–40 cells and filters 50–300 candidates by freshness and availability.
 - Decision: ETA comes from the routing service only for the top 10 candidates by straight-line distance.
 - Routing calls stay near 12k/s at global peak instead of 300k/s.
 - Cache ETAs per cell pair for a few seconds where precision allows.
- Offer rounds
 - A 15 s offer deadline with an assumed average of 1.5 rounds gives a typical assignment in 15–25 s.
 - 5 rounds bound the worst case near 75–90 s before NO_DRIVERS.
 - Decision: rounds and deadline are per-tenant configuration, not code constants.
- Trip state
 - 25M trips/day at about 1 KB per trip is 25 GB/day, on the order of 9 TB/year.
 - Active trips at any time are a few hundred thousand rows.
 - Decision: the hot relational store holds active and recent trips per region.
 - Archive completed trips by tenant and month to a cheaper store.
- Live-location fan-out
 - An assumed 60% of 1M online drivers on a trip is about 600k active trips at peak.
 - Each active trip has one rider subscriber at 1 update/s, so fan-out is about 600k messages/s.
 - Decision: fan-out is a pub/sub topic per trip behind websocket gateways.
 - The rider-facing rate is capped at 1/s regardless of driver update rate.

Architecture and flow — IC3 baseline



- Location gateway owns admission and fencing of driver updates.
 - Mechanism bought: authenticates the driver session, rejects stale (session_epoch, seq), and forwards only monotonic updates.
 - Cost paid: every update carries a session identity, and a restarted app must obtain a new epoch before its updates count.
- Latest-location store owns the nearest-driver index.
 - Mechanism bought: an in-memory map keyed by driver plus a per-cell membership set, with a 30 s freshness TTL.
 - Cost paid: it is a projection with no durability promise; a node loss forgets drivers until their next update arrives.

- Matcher owns candidate selection and offers for one region or cell lease.
 - Mechanism bought: a single logical writer per trip during matching, so offers are sequential and rounds are bounded.
 - Cost paid: a lease holder can crash mid-round; the offer deadline and lease expiry are what bound the stall.
- Trip API and trip store own authoritative trip and driver state.
 - Mechanism bought: a relational store with versioned rows, a partial unique index on active trips per driver, and an outbox in the same transaction.
 - Cost paid: single-region writes; a store outage stops new assignments even though ingest and in-progress trips continue.
- Routing service is a dependency, not a component of this design.
 - Mechanism bought: ETA ranking that beats straight-line distance in cities.
 - Cost paid: a synchronous call on the matching path; fall back to haversine ranking when it is slow.
- Pub/sub and websocket gateway own live-location delivery.
 - Mechanism bought: one topic per active trip so authorization is a topic subscription check.
 - Cost paid: connection state on the gateway; a gateway loss forces reconnect and resubscribe.

→ Rider calls `POST /ride-quotes`; the quote is priced server-side with a bounded `expires_at`.

→ Rider calls `POST /ride-requests` with the quote and an `Idempotency-Key`; the trip row is created in `MATCHING` and returns `202`.

→ The matcher for the pickup cell takes the trip, computes the cell cover for the search radius, and reads candidates from the latest-location store.

→ Candidates are filtered by freshness and by the availability projection, then the top 10 by straight-line distance are ranked by routing ETA.

→ The matcher inserts one offer for the best driver with a 15 s deadline and a deterministic `offer_id`, then pushes it to the driver app.

→ The driver calls `POST /trip-offers/{id}/acceptances`; the transaction verifies the offer, the trip version, and the driver's `AVAILABLE` state, then claims both.

→ The commit writes an outbox event; the rider is notified and subscribed to the trip's location topic.

→ The driver transitions the trip through `ARRIVING`, `IN_PROGRESS`, and `COMPLETED` with `expectedVersion`; completion emits a settlement event keyed by `trip_id`.

API and data model — IC3 baseline

- Paths omit the `/v1` prefix and follow the [shared API conventions](#).
 - `I` means `Idempotency-Key` is required.
 - `V` means a strong `If-Match` validator is required.

Surface	Contract	Idempotency	Conflict/error behavior
<code>POST /ride-quotes</code>	<code>{pickup, dropoff, rideType};</code> returns <code>201 {quoteId, amountMinor, currency, expiresAt}</code>	Read-like; a new quote per call	<code>422</code> on unserviceable area
<code>POST /ride-requests</code>	<code>{quoteId, pickup, dropoff};</code> returns <code>202 {tripId, state: MATCHING}</code>	<code>I</code> per rider	<code>409</code> on an expired quote or an existing active trip for the rider
<code>PUT /drivers/{id}/location</code>	<code>{sessionEpoch, seq, lat, lon, capturedAt};</code> returns <code>200 {acceptedSeq}</code>	Naturally idempotent by <code>(epoch, seq)</code>	Stale <code>(epoch, seq)</code> returns <code>200</code> with the higher accepted seq and is not applied
<code>PUT /drivers/{id}/availability</code>	<code>{state: AVAILABLE OFFLINE};</code> returns <code>200 driver</code>	<code>V</code>	<code>409</code> when an active trip owns the driver

Surface	Contract	Idempotency	Conflict/error behavior
POST /trip-offers/{id}/acceptances	{expectedTripVersion}; returns 200 trip	I per driver	409 OFFER_LOST when the offer expired, the trip moved on, or the driver is no longer AVAILABLE
POST /trip-offers/{id}/declines	{reason}; returns 204	Idempotent by offer state	409 if already accepted
POST /trips/{id}/transitions	{expectedVersion, event}; returns 200 trip	I per actor	409 on version mismatch or an event the actor may not send
POST /trips/{id}/cancellations	{reason, expectedVersion}; returns 200 or 202 cancellation	I	409 if the trip is IN_PROGRESS or terminal; fee rule depends on state
GET /trips/{id}	Authorized trip view	Read	404 for a trip the caller does not own or drive
GET /trips/{id}/driver-location	Latest assigned-driver location, or a stream subscription token	Read	404 unless the caller is the rider or driver of an active trip

State	Key	Fields/invariant
Quote	(tenant_id, quote_id)	Rider, amount in minor units, currency, route hash, expires_at
Driver	(tenant_id, driver_id)	State OFFLINE\ AVAILABLE\ BUSY, version, current session_epoch
Trip	(tenant_id, trip_id)	Rider, quote, driver, state, version, pickup, dropoff, created_at; partial unique index on (tenant_id, driver_id) while active
Offer	(tenant_id, offer_id)	Trip, driver, round, expires_at, state OFFERED\ ACCEPTED\ DECLINED\ EXPIRED\ LOST
Latest location	(tenant_id, driver_id) in memory	session_epoch, seq, lat, lon, received_at, cell; TTL 30 s
Cell membership	(tenant_id, region, cell) in memory	Set of driver IDs with freshness metadata
Location history	(tenant_id, driver_id, time_bucket)	Append-only points under retention
Idempotency record	(tenant_id, caller_id, operation, intent_key)	Request hash and stored response; retained beyond the retry horizon
Outbox	(tenant_id, event_id)	Aggregate, version, payload; inserted in the same transaction as the state change

- Reference DDL (the companion holds the full schema; this adds the offer fields the matcher needs)

```
CREATE TABLE trip_offers (
  tenant_id text NOT NULL, offer_id text NOT NULL,      -- offer_id = hash(trip_id, driver_id, round)
  trip_id   text NOT NULL, driver_id text NOT NULL,
  round     integer NOT NULL, expires_at timestamptz NOT NULL,
  state     text NOT NULL,                            -- OFFERED|ACCEPTED|DECLINED|EXPIRED|LOST
  PRIMARY KEY (tenant_id, offer_id),
  UNIQUE (tenant_id, trip_id, round),                 -- one offer per round
  FOREIGN KEY (tenant_id, trip_id) REFERENCES trips,
  FOREIGN KEY (tenant_id, driver_id) REFERENCES ride_drivers
);
CREATE UNIQUE INDEX driver_one_trip ON trips (tenant_id, driver_id)
  WHERE state IN ('ASSIGNED', 'ARRIVING', 'IN_PROGRESS');
```

- Conventions
 - tenant_id and the actor come from authenticated context, never from the body.

- `expectedVersion` in the body is a domain precondition on the trip, so its failure is 409, not 412.
- A stale location update returns 200 rather than an error so that a driver app never retries a rejected point.
- Money is minor units plus currency; timestamps carry offsets.

Location ingest and geo index — IC3 baseline

- Cell choice
 - Geohash precision 6 is a cell of about $1.2 \text{ km} \times 0.6 \text{ km}$ at the equator; that is the assumed index resolution.
 - S2 level 13 or H3 resolution 8 are the same order of magnitude and avoid geohash's edge-length distortion at high latitudes.
 - The interview claim is "a hierarchical cell index at roughly 1 km"; the library is a knob.
 - Cell size trades index selectivity against the number of cells a search radius touches.
- Index structure
 - `driver -> (epoch, seq, lat, lon, cell, received_at)` is the latest-location map.
 - `cell -> set(driver)` is the membership index.
 - A location update that changes cell removes the driver from the old cell and adds it to the new one under the driver's lock.
 - Freshness uses `received_at` at the server, not the device's `capturedAt`, so device clock skew cannot keep a stale entry alive.
- Search
 - Compute the set of cells that intersect the circle of the search radius around the pickup.
 - Query all intersecting cells; a fixed 9-neighbor query is only correct when the radius is smaller than one cell.
 - Filter candidates by `received_at` within 30 s and by the availability projection.
 - Rank by straight-line distance, then re-rank the top 10 by routing ETA.
 - Widen the radius per round, for example 2 km, 4 km, 8 km, until the tenant's cap.
- Fencing

```
APPLY_LOCATION(driver, update):
    lock driver entry
    if update.epoch < entry.epoch: return accepted_seq = entry.seq          # old session
    if update.epoch == entry.epoch and update.seq <= entry.seq: return entry.seq # reordered or replayed
    move driver between cells if the cell changed
    entry = (update.epoch, update.seq, update.lat, update.lon, cell, now())
    publish to trip topic if the driver is on an active trip (rate-capped to 1/s)
    return entry.seq
```

```
- `session_epoch` is issued by the availability endpoint when the driver goes online; an app restart gets a new, higher epoch.
- `seq` increases per session on the device; a lost response is retried with the same `seq`.
- Without the fence, two gateways receiving updates out of order can move a driver backward and the matcher offers a trip to a driver who has already left.
```

- Consistency boundary
 - The latest-location store is eventually consistent and lossy by design.
 - It answers "who is probably near here right now", never "who is assigned".
 - Availability in the index is a projection of the driver row; the trip transaction re-reads the driver row.
- Concrete values
 - 30 s TTL is about 7 missed updates at a 4 s interval; shorter TTLs drop drivers on brief signal loss, longer TTLs offer trips to drivers who are gone.
 - Per-region store shards by cell prefix; the busiest metro spreads across several nodes so one node does not carry 25k/s.
- Exact failure

- A store node is lost.
- Its drivers vanish from the index for up to one update interval, then reappear as updates arrive.
- Matching in those cells widens its radius for a few seconds; nothing is assigned incorrectly because assignment never reads the index as truth.

Matching, offers, and the trip state machine — IC3 baseline

```
MATCHING --accept commits--> ASSIGNED --driver arrives--> ARRIVING --pickup--> IN_PROGRESS --dropoff-->
COMPLETED
MATCHING --rounds exhausted--> NO_DRIVERS
MATCHING|ASSIGNED|ARRIVING --rider or driver cancel--> CANCELLED (fee rule depends on the state it left)
```

- Offer lifecycle
 - The matcher holds a lease on the trip's pickup cell or region, so one logical writer issues offers for a trip.
 - `offer_id = hash(trip_id, driver_id, round)` and `UNIQUE (tenant_id, trip_id, round)` make an offer insert idempotent.
 - A matcher retry after a crash reads the existing offer for the round instead of issuing a second one.
 - The offer carries `expires_at = now + 15 s`; the push to the driver is best effort and the driver app can also poll its pending offer.
 - On decline or expiry, the matcher moves to the next candidate and increments the round.
 - After 5 rounds or 90 s, the trip transitions to `NO_DRIVERS` with the same expected-version discipline.

- Accept transaction

```
ACCEPT(tenant, offer_id, driver, expected_trip_version):
  transaction:
    offer = select trip_offers for update where offer_id
    require offer.driver_id == driver and offer.state == OFFERED and now() < offer.expires_at
    trip = select trips for update where trip_id = offer.trip_id
    require trip.state == MATCHING and trip.version == expected_trip_version
    drv = select ride_drivers for update where driver_id
    require drv.state == AVAILABLE
    update trips set state = ASSIGNED, driver_id = driver, version = version + 1
    update ride_drivers set state = BUSY, version = version + 1
    update trip_offers set state = ACCEPTED where offer_id
    insert outbox (TripAssigned, trip_id, version)
  on any require failure: return 409 OFFER_LOST
  on unique violation of driver_one_trip: return 409 OFFER_LOST
```

```
- Lock order is offer, then trip, then driver, in every code path, so two accepts cannot deadlock.
- The partial unique index is the second line of defense: if a bug ever offers one driver two trips, the
  second commit fails instead of double-booking.
- The driver's `Idempotency-Key` replays the stored `200` on a retried accept, so a lost response does not
  surface as `OFFER_LOST`.
```

- Concurrency cases
 - Two riders, one driver: only when broadcast offers are enabled can both offers exist; the first commit wins and the second returns `409 OFFER_LOST`.
 - Rider cancels while an offer is outstanding: cancellation moves the trip to `CANCELLED` with a version bump, so the accept fails its version check.
 - Driver goes `OFFLINE` while holding an offer: the availability endpoint is `V`-guarded and the accept re-reads `AVAILABLE` under lock.
 - Offer expires during the accept: the deadline check runs inside the transaction using the database clock.
- Transition rules
 - Each event names the actors allowed to send it; `ARRIVING`, `IN_PROGRESS`, and `COMPLETED` are driver-only, `CANCELLED` is rider or driver with different fee outcomes.

- Every transition is `UPDATE ... WHERE version = expected` and inserts an outbox row in the same transaction.
- A retried transition with the same `Idempotency-Key` replays the stored response.
- Completion and settlement
 - `COMPLETED` writes a `TripCompleted` outbox event with the final fare inputs.
 - The settlement service charges once per `trip_id` with its own idempotent record and compensates on failure.
 - Payment failure never reopens the trip; it creates a receivable to resolve out of band.
- Static stability
 - The driver app holds the trip state and the next allowed transitions locally.
 - If the trip store is unavailable, the driver continues the ride and the app queues `IN_PROGRESS` and `COMPLETED` with their idempotency keys.
 - Location ingest and the rider's live map keep working because they never touch the trip store.
 - New ride requests fail fast with `503` rather than queueing indefinitely, because a rider waiting for a driver that cannot be assigned is worse than an honest error.
- Exact failure
 - The matcher crashes after inserting an offer and before pushing it.
 - The lease expires, a new matcher reads the round's existing offer, and re-pushes it if time remains.
 - If the offer already expired, the new matcher moves to the next round; the driver app ignores an expired offer by `expires_at`.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Latest-location store node lost	Drivers in that shard's cells for one update interval	Shard health; candidate count drop per cell	Matching widens radius; no incorrect assignment	Entries rebuild from live updates within <code>30 s</code>
Location updates reordered or replayed	One driver	Rejected-seq counter	Stale update ignored, <code>200</code> returned	None needed; fence is the correction
Driver loses connectivity while <code>AVAILABLE</code>	One driver	TTL expiry	Driver drops out of candidates after <code>30 s</code> ; an outstanding offer expires	Driver reconnects with a new epoch
Matcher crash mid-round	Trips in one lease	Lease expiry; oldest <code>MATCHING</code> age	Round stalls up to the lease TTL	New lease holder resumes from the persisted offer
Push notification lost	One offer	Driver app poll; offer expiry	Driver polls pending offers; expiry moves on	Next round
Two accepts race for one driver	One driver, two trips	<code>409 OFFER_LOST</code> rate	Second accept rejected	Matcher offers the losing trip elsewhere
Routing/ETA service slow	Match quality in one region	Call latency, timeout rate	Rank by straight-line distance; ETA cache	Re-enable ETA ranking when latency recovers
Trip store unavailable	New requests and transitions in one region	Health check; error rate	In-progress trips continue on device; ingest and live map work; new requests get <code>503</code>	Fail over the replica; devices replay queued transitions by idempotency key
Settlement service down	Payments, not trips	Outbox lag	Trips complete; charges queue	Outbox relay retries; reconcile receivables
Region boundary trip	One trip crossing regions	Driver cell outside home region	Home region keeps trip ownership; location routes by driver, not by cell	Handoff only for a new trip after completion

- Observe
 - Location updates per second, rejected-seq rate, and per-cell candidate counts.

- Time to first offer, time to assignment, and rounds per trip by tenant.
- Offer expiry rate and `OFFER_LOST` rate.
- Oldest `MATCHING` trip age per region.
- Outbox lag and settlement retry depth.
- Live-location delivery latency and gateway connection churn.
- Retry discipline
 - Driver location updates retry the same `(epoch, seq)` with a short deadline, then skip to the newest point.
 - Accept, transition, and cancellation retries reuse the same `Idempotency-Key` until a terminal answer arrives.
 - The matcher never retries an offer insert with a new `offer_id`; it re-reads the round.
 - Routing calls have a hard timeout of a few hundred milliseconds and a haversine fallback, not a retry loop.
- Tenant isolation
 - Every key starts with `tenant_id`; the cell index is namespaced by tenant so a search never returns another operator's drivers.
 - Per-tenant quotas cap ride requests per second and online drivers.
 - A caller receives `404`, not `403`, for another tenant's trip or driver IDs.
 - Matching configuration, such as radius steps and offer deadline, is per tenant.
- Rollout and rollback safety
 - Trip-state schema changes follow expand, migrate, contract; a new transition rule must accept trips created by the previous version.
 - Canary the matcher by region; a bad ranking change affects one region's match quality, not assignment correctness.
 - Location gateway deploys are rolling; the in-memory store is rebuilt from live updates, so a node restart costs one update interval.
 - Driver-app and API versions overlap; an unknown event from a newer app returns `400`, never a silent state change.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Geohash cells	Base-32 prefix cells in an ordered key space	Simple; prefix range scans on any sorted store	Cell shape distorts by latitude; neighbor logic at edges	Baseline with a sorted KV or in-memory sets
S2 or H3 cells	Hierarchical spherical cells (quad or hex)	Uniform-ish area; region cover for a radius is a library call	One more dependency; hex has no exact parent containment	Global service where latitude distortion matters
Redis GEOSEARCH	Sorted-set geohash with radius queries	Ready-made radius query with no index code to write	No per-member TTL, so freshness needs a side structure; single-key hot spot per region; still not an assignment	Small deployments or a prototype
PostGIS or R-tree	Spatial index inside the database	Exact geometry; joins with trip data	Cannot absorb 250k writes/s	Low-rate fleets or analytics
Sequential offers	One driver at a time with a deadline	No double-offer race; simple fairness	Slower assignment under declines	Baseline
Broadcast offers	Offer to top <code>k</code> at once	Faster acceptance	<code>k-1</code> losers; needs <code>OFFER_LOST</code> handling	Low supply or high decline rates
Batched assignment	Solve riders-to-drivers per cell every 2–5 s	Better global ETA; fewer offers	Latency floor; solver complexity	Dense metros at peak

- IC4 stretch — batched matching
 - Collect `MATCHING` trips per cell for a 2–5 s window and solve a small assignment problem on ETA.
 - The accept transaction does not change; the batch only decides who gets offered first.
 - Cost: a latency floor per window and a solver that must be bounded per cell.

- IC4 stretch — 10× load
 - 2.5M updates/s still fits in memory per region; shard cells across more nodes and replicate each shard.
 - Matching at 12k/s global peak needs ETA caching per cell pair and routing calls capped per region.
 - Move the location log to a partitioned append log with per-region consumers; history stays off the hot path.
 - Split the trip store by region and then by tenant when a single relational store's write rate is the limit.
- IC4 stretch — cost and blast radius
 - The in-memory store and websocket gateways dominate cost; the trip store is small.
 - Cell the deployment per region so a region's matcher, store, and gateway fail together and alone.
 - Degrade by widening radius and dropping ETA ranking before dropping matching entirely.
- IC4 stretch — pooling and region handoff
 - Pooling changes the invariant to one active trip per driver with many riders; the partial unique index becomes a capacity check.
 - Cross-region trips keep ownership in the home region; only new trips after completion move.
- Weakest assumption
 - Driver location updates arrive at 4 s intervals with independent losses.
 - A carrier outage makes a whole neighborhood stale at once, so the index shows drivers who are all unreachable and every offer expires.
 - Mitigation: treat the offer itself as a liveness probe, shorten the TTL during high expiry rates, and alarm on per-cell offer-expiry spikes.
- Over-engineering warning
 - Do not build a global assignment optimizer before sequential offers with a bounded radius are measured.
 - Do not make the geo index strongly consistent; correctness lives in the trip transaction.
 - Do not call the routing service for every candidate.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Online drivers, update interval, one-driver-one-trip invariant, what survives a trip-store outage	Names pooling and batched matching as non-goals and what cutting them buys	Negotiates the matching policy instead of accepting it
Estimates	250k/s ingest, 1.2k/s matches, 600k/s fan-out	Lets the ingest rate rule out the relational store and the match rate cap routing calls	A number changes a decision
Architecture	Location data plane versus trip control plane with one trace	States the cost of each split: a lossy projection and a single-region trip store	Every component has a mechanism and a price
Deep dive	Cell cover query, (epoch, seq) fence, accept transaction with lock order and unique index	Broadcast and batched offers with OFFER_LOST; leases and idempotent offer rounds	Reasons across the index and the transaction boundary
Operations	Failure table, degraded matching, static stability of in-progress trips	Per-region cells, tenant quotas, rollout of ranking changes	Volunteers degradation and rollout before being asked
Wrap	Recap and tradeoff	Names correlated staleness as the weakest assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why not query drivers within a radius directly from the database?
 - **IC3 baseline**
 - 250k writes/s and a distance scan per request do not fit a relational store.
 - The index is a projection; the database is for assignment.

- **IC4 stretch**
 - PostGIS is right for fleets of thousands, and saying where the crossover lies is the signal.
- Q2: Two riders request at the same time and the nearest driver is the same.
 - **IC3 baseline**
 - Sequential offers per driver mean only one trip holds an offer for that driver at a time.
 - The accept transaction and the partial unique index reject any second claim.
 - **IC4 stretch**
 - Under broadcast offers, the loser gets `409 OFFER_LOST` and its matcher moves on within the same round.
- Q3: A driver's phone reconnects and replays old location updates.
 - **IC3 baseline**
 - The `(session_epoch, seq)` fence drops anything not newer than the stored value.
 - **IC4 stretch**
 - Server receipt time drives freshness, so a device clock cannot resurrect a stale entry.
- Q4: The trip database goes down for 20 minutes.
 - **IC3 baseline**
 - In-progress trips continue on the device; ingest and the live map keep working; new requests get `503`.
 - **IC4 stretch**
 - Queued `COMPLETED` transitions replay by idempotency key, and settlement waits on the outbox rather than on the outage.
- Q5: How do you guarantee an offer is not sent twice after a matcher crash?
 - **IC3 baseline**
 - `offer_id = hash(trip_id, driver_id, round)` and the unique round constraint make the insert idempotent.
 - **IC4 stretch**
 - The lease holder re-reads the round on takeover and re-pushes only if the deadline has not passed.
- Q6: Now add ride pooling.
 - **IC3 baseline**
 - The one-trip-per-driver invariant becomes a seat-capacity check inside the same transaction.
 - **IC4 stretch**
 - Matching becomes an insertion problem against an active route, which is where batched assignment earns its cost.
- Q7 (premise reversal): Why push offers at all instead of letting drivers poll for nearby requests and claim one?
 - **IC3 baseline**
 - Driver-pull turns every request into a race among many drivers, so most claims lose and the rider waits on contention rather than on ETA.
 - The claim transaction would still be needed, so pull buys nothing on correctness.
 - **IC4 stretch**
 - Pull is reasonable for low-density markets with few requests; then the claim is the same accept transaction with a first-come rule and the cell index serves drivers instead of riders.
- Q8: The interviewer asks for live locations of all nearby drivers on the rider's map before the request.
 - **IC3 baseline**
 - Serve a coarse, cell-level snapshot with reduced precision and no driver identity, at a low refresh rate.
 - **IC4 stretch**
 - Precise per-driver positions to arbitrary clients are a privacy and scraping problem; the assigned-driver-only topic stays the only precise feed.
- Validation scenarios
 - Two accepts for one driver within the same second.
 - Rider cancels while an offer is outstanding.
 - Location update with an older epoch after an app restart.

- Matcher crash between offer insert and push.
- Trip store unavailable during `IN_PROGRESS`, then `COMPLETED` replayed twice.
- Search radius larger than one cell at the cell edge.
- Routing service timeout during ranking.
- Tenant A token on tenant B trip ID returns `404`.

Recall card

- “The geo index nominates; the trip transaction assigns.”
- “Fence every location update by session epoch and sequence, and expire it by server time.”
- “One offer per round with a deterministic ID and a deadline; the accept claims trip and driver together under a unique index.”
- “In-progress trips must survive a trip-store outage; new requests should fail fast.”
- “Cover the radius with cells; never assume nine neighbors.”
- Wrap-up script
 - “The design is a fenced location data plane feeding a cell index, a per-region matcher issuing timed idempotent offers, and a transactional trip service with a versioned state machine.”
 - “The key invariant is that a driver has at most one active trip, enforced by one conditional accept transaction and a partial unique index, never by the index.”
 - “The main tradeoff is a lossy in-memory location projection: it absorbs `250k` writes per second and rebuilds itself, in exchange for candidates that are sometimes stale and must be re-verified by the offer.”
 - “My weakest assumption is independent location loss; a correlated carrier outage stales a whole neighborhood, so I would validate offer-expiry alarms and adaptive TTL first.”

References

- [Ranked question and report provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- [Shared API conventions](#): `I` and `V` semantics, `202` operations, error shapes.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values, cell resolution, TTLs, offer deadlines, and round limits remain proposed interview assumptions.
- [H3: Uber's hexagonal hierarchical spatial index](#): hierarchical cell indexing for supply and demand; hex neighbor uniformity.
- [S2 geometry](#): hierarchical spherical cells and region covering for radius queries.
- [Geohash](#): prefix cells, precision table, and edge-case limitations.
- [Redis GEOSEARCH](#): sorted-set radius query used as the prototype alternative.
- [PostgreSQL partial indexes](#): the `driver_one_trip` unique index on active trips.
- [PostgreSQL row locking](#): `SELECT ... FOR UPDATE` in the accept transaction.
- [Transactional outbox](#): assignment and completion events in the same transaction.
- Sibling designs
 - [s1.05 metrics ingestion](#): high-rate ingest and async history.
 - [s3.05 leases and locks](#): matcher lease semantics.
 - [s4.02 chat](#): websocket gateway and fan-out.